

Sequence-to-graph alignment acceleration on CPU-GPU systems

What is sequence alignment?

Problem definition

- In bioinformatics, a sequence alignment is a way of arranging the sequences of DNA, RNA, or protein to identify regions of similarity that may be a consequence of functional, structural, or evolutionary relationships between the sequences.

AATCGTAGGCC
AAATAGGCC
AAACTACGCC
AAGCTAGGCG
AAACGTAAGCC



A	A	T	C	G	T	A	G	G	C	C
A	A	A	-	-	T	A	G	G	C	C
A	A	A	C	-	T	A	C	G	C	C
A	A	G	C	-	T	A	G	G	C	G
A	A	A	C	G	T	A	A	G	C	C

What is sequence alignment?

Smith-Waterman algorithm

The Smith-Waterman algorithm

$$H_{ij} = \max \begin{cases} H_{i-1,j-1} + s(a_i, b_j), \\ \max_{k \geq 1} \{H_{i-k,j} - W_k\}, \\ \max_{l \geq 1} \{H_{i,j-l} - W_l\}, \\ 0 \end{cases} \quad (1 \leq i \leq n, 1 \leq j \leq m)$$

where

$H_{i-1,j-1} + s(a_i, b_j)$ is the score of aligning a_i and b_j ,

$H_{i-k,j} - W_k$ is the score if a_i is at the end of a gap of length k ,

$H_{i,j-l} - W_l$ is the score if b_j is at the end of a gap of length l ,

0 means there is no similarity up to a_i and b_j .

	T	G	T	T	A	C	G	G
G								
G								
T								
T								
G								
A								
C								
T								
A								

What is sequence alignment?

Smith-Waterman algorithm

The Smith-Waterman algorithm

$$H_{ij} = \max \begin{cases} H_{i-1,j-1} + s(a_i, b_j), \\ \max_{k \geq 1} \{H_{i-k,j} - W_k\}, \\ \max_{l \geq 1} \{H_{i,j-l} - W_l\}, \\ 0 \end{cases} \quad (1 \leq i \leq n, 1 \leq j \leq m)$$

where

$H_{i-1,j-1} + s(a_i, b_j)$ is the score of aligning a_i and b_j ,

$H_{i-k,j} - W_k$ is the score if a_i is at the end of a gap of length k ,

$H_{i,j-l} - W_l$ is the score if b_j is at the end of a gap of length l ,

0 means there is no similarity up to a_i and b_j .

	T	G	T	T	A	C	G	G
	0	0	0	0	0	0	0	0
G	0							
G	0							
T	0							
T	0							
G	0							
A	0							
C	0							
T	0							
A	0							

What is sequence alignment?

Smith-Waterman algorithm

The Smith-Waterman algorithm

$$H_{ij} = \max \begin{cases} H_{i-1,j-1} + s(a_i, b_j), \\ \max_{k \geq 1} \{H_{i-k,j} - W_k\}, \\ \max_{l \geq 1} \{H_{i,j-l} - W_l\}, \\ 0 \end{cases} \quad (1 \leq i \leq n, 1 \leq j \leq m)$$

where

$H_{i-1,j-1} + s(a_i, b_j)$ is the score of aligning a_i and b_j ,

$H_{i-k,j} - W_k$ is the score if a_i is at the end of a gap of length k ,

$H_{i,j-l} - W_l$ is the score if b_j is at the end of a gap of length l ,

0 means there is no similarity up to a_i and b_j .

		T	G	T	T	A	C	G	G
	0	0	0	0	0	0	0	0	0
G	0	0							
G	0								
T	0								
T	0								
G	0								
A	0								
C	0								
T	0								
A	0								

What is sequence alignment?

Smith-Waterman algorithm

The Smith-Waterman algorithm

$$H_{ij} = \max \begin{cases} H_{i-1,j-1} + s(a_i, b_j), \\ \max_{k \geq 1} \{H_{i-k,j} - W_k\}, \\ \max_{l \geq 1} \{H_{i,j-l} - W_l\}, \\ 0 \end{cases} \quad (1 \leq i \leq n, 1 \leq j \leq m)$$

where

$H_{i-1,j-1} + s(a_i, b_j)$ is the score of aligning a_i and b_j ,

$H_{i-k,j} - W_k$ is the score if a_i is at the end of a gap of length k ,

$H_{i,j-l} - W_l$ is the score if b_j is at the end of a gap of length l ,

0 means there is no similarity up to a_i and b_j .

		T	G	T	T	A	C	G	G
	0	0	0	0	0	0	0	0	0
G	0	0	3						
G	0								
T	0								
T	0								
G	0								
A	0								
C	0								
T	0								
A	0								

What is sequence alignment?

Smith-Waterman algorithm

The Smith-Waterman algorithm

$$H_{ij} = \max \begin{cases} H_{i-1,j-1} + s(a_i, b_j), \\ \max_{k \geq 1} \{H_{i-k,j} - W_k\}, \\ \max_{l \geq 1} \{H_{i,j-l} - W_l\}, \\ 0 \end{cases} \quad (1 \leq i \leq n, 1 \leq j \leq m)$$

where

$H_{i-1,j-1} + s(a_i, b_j)$ is the score of aligning a_i and b_j ,

$H_{i-k,j} - W_k$ is the score if a_i is at the end of a gap of length k ,

$H_{i,j-l} - W_l$ is the score if b_j is at the end of a gap of length l ,

0 means there is no similarity up to a_i and b_j .

	T	G	T	T	A	C	G	G
G	0	0	0	0	0	0	0	0
G	0	0	3	1	0	0	0	3
T	0							
T	0							
G	0							
A	0							
C	0							
T	0							
A	0							

What is sequence alignment?

Smith-Waterman algorithm

The Smith-Waterman algorithm

$$H_{ij} = \max \begin{cases} H_{i-1,j-1} + s(a_i, b_j), \\ \max_{k \geq 1} \{H_{i-k,j} - W_k\}, \\ \max_{l \geq 1} \{H_{i,j-l} - W_l\}, \\ 0 \end{cases} \quad (1 \leq i \leq n, 1 \leq j \leq m)$$

where

$H_{i-1,j-1} + s(a_i, b_j)$ is the score of aligning a_i and b_j ,

$H_{i-k,j} - W_k$ is the score if a_i is at the end of a gap of length k ,

$H_{i,j-l} - W_l$ is the score if b_j is at the end of a gap of length l ,

0 means there is no similarity up to a_i and b_j .

		T	G	T	T	A	C	G	G
	0	0	0	0	0	0	0	0	0
G	0	0	3	1	0	0	0	3	3
G	0	→0	→3	→1	→0	→0	→0	→3	→6
T	0								
T	0								
G	0								
A	0								
C	0								
T	0								
A	0								

What is sequence alignment?

Smith-Waterman algorithm

The Smith-Waterman algorithm

$$H_{ij} = \max \begin{cases} H_{i-1,j-1} + s(a_i, b_j), \\ \max_{k \geq 1} \{H_{i-k,j} - W_k\}, \\ \max_{l \geq 1} \{H_{i,j-l} - W_l\}, \\ 0 \end{cases} \quad (1 \leq i \leq n, 1 \leq j \leq m)$$

where

$H_{i-1,j-1} + s(a_i, b_j)$ is the score of aligning a_i and b_j ,

$H_{i-k,j} - W_k$ is the score if a_i is at the end of a gap of length k ,

$H_{i,j-l} - W_l$ is the score if b_j is at the end of a gap of length l ,

0 means there is no similarity up to a_i and b_j .

	T	G	T	T	A	C	G	G
G	0	0	0	0	0	0	0	0
G	0	0	3	1	0	0	0	3
G	0	0	3	1	0	0	0	3
T	0	3	1	6	4	2	0	1
T	0							
G	0							
A	0							
C	0							
T	0							
A	0							

What is sequence alignment?

Smith-Waterman algorithm

The Smith-Waterman algorithm

$$H_{ij} = \max \begin{cases} H_{i-1,j-1} + s(a_i, b_j), \\ \max_{k \geq 1} \{H_{i-k,j} - W_k\}, \\ \max_{l \geq 1} \{H_{i,j-l} - W_l\}, \\ 0 \end{cases} \quad (1 \leq i \leq n, 1 \leq j \leq m)$$

where

$H_{i-1,j-1} + s(a_i, b_j)$ is the score of aligning a_i and b_j ,

$H_{i-k,j} - W_k$ is the score if a_i is at the end of a gap of length k ,

$H_{i,j-l} - W_l$ is the score if b_j is at the end of a gap of length l ,

0 means there is no similarity up to a_i and b_j .

	T	G	T	T	A	C	G	G
G	0	0	3	1	0	0	3	3
G	0	0	3	1	0	0	3	6
T	0	3	1	6	4	2	0	4
T	0	3	1	4	9	7	5	3
G	0	1	6	4	7	6	4	8
A	0	0	4	3	5	10	8	6
C	0	0	2	1	3	8	13	9
T	0	3	1	5	4	6	11	10
A	0	1	0	3	2	7	9	8

What is sequence alignment?

Smith-Waterman algorithm

The Smith-Waterman algorithm

$$H_{ij} = \max \begin{cases} H_{i-1,j-1} + s(a_i, b_j), \\ \max_{k \geq 1} \{H_{i-k,j} - W_k\}, \\ \max_{l \geq 1} \{H_{i,j-l} - W_l\}, \\ 0 \end{cases} \quad (1 \leq i \leq n, 1 \leq j \leq m)$$

where

$H_{i-1,j-1} + s(a_i, b_j)$ is the score of aligning a_i and b_j ,

$H_{i-k,j} - W_k$ is the score if a_i is at the end of a gap of length k ,

$H_{i,j-l} - W_l$ is the score if b_j is at the end of a gap of length l ,

0 means there is no similarity up to a_i and b_j .

	T	G	T	T	A	C	G	G
G	0	0	0	0	0	0	0	0
G	0	0	3	1	0	0	0	3
T	0	3	1	6	4	2	0	1
T	0	3	1	4	9	7	5	3
G	0	1	6	4	7	6	4	8
A	0	0	4	3	5	10	8	6
C	0	0	2	1	3	8	13	9
T	0	3	1	5	4	6	11	10
A	0	1	0	3	2	7	9	8

What is sequence alignment?

Smith-Waterman algorithm

The Smith-Waterman algorithm

$$H_{ij} = \max \begin{cases} H_{i-1,j-1} + s(a_i, b_j), \\ \max_{k \geq 1} \{H_{i-k,j} - W_k\}, \\ \max_{l \geq 1} \{H_{i,j-l} - W_l\}, \\ 0 \end{cases} \quad (1 \leq i \leq n, 1 \leq j \leq m)$$

where

$H_{i-1,j-1} + s(a_i, b_j)$ is the score of aligning a_i and b_j ,

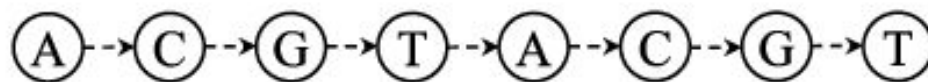
$H_{i-k,j} - W_k$ is the score if a_i is at the end of a gap of length k ,

$H_{i,j-l} - W_l$ is the score if b_j is at the end of a gap of length l ,

0 means there is no similarity up to a_i and b_j .

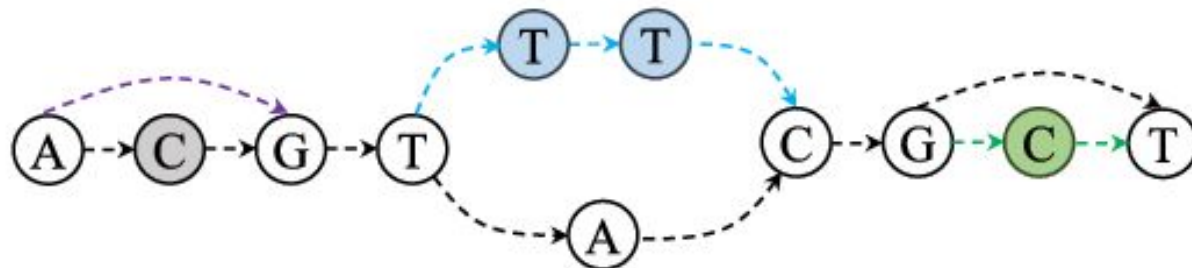
		T	G	T	T	A	C	G	G
	0	0	0	0	0	0	0	0	0
G	0	0	3	1	0	0	0	3	3
G	0	0	3	1	0	0	0	3	6
T	0	3	1	6	4	2	0	1	4
T	0	3	1	4	9	7	5	3	2
G	0	1	6	4	7	6	4	8	6
A	0	0	4	3	5	10	8	6	5
C	0	0	2	1	3	8	13	11	9
T	0	3	1	5	4	6	11	10	8
A	0	1	0	3	2	7	9	8	7

**Linear
reference**



- Deletion
- Insertion
- Substitution

**Graph
reference**



Sequence-to-graph alignment

How do we adapt the algorithm?

Partial Order Alignment

Partial Order Alignment (POA)

$$H_{ij} = \max \begin{cases} H_{i-1,j-1} + s(a_i, b_j), \\ \max_{k \geq 1} \{H_{i-k,j} - W_k\}, \\ \max_{l \geq 1} \{H_{i,j-l} - W_l\}, \\ 0 \end{cases} \quad (1 \leq i \leq n, 1 \leq j \leq m)$$

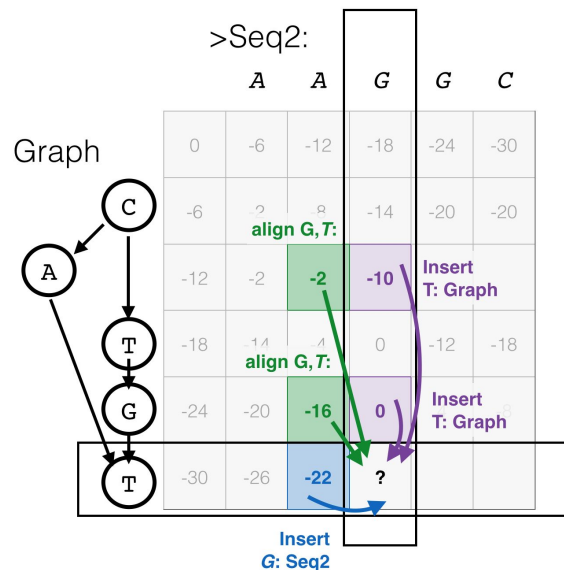
where

$H_{i-1,j-1} + s(a_i, b_j)$ is the score of aligning a_i and b_j ,

$H_{i-k,j} - W_k$ is the score if a_i is at the end of a gap of length k ,

$H_{i,j-l} - W_l$ is the score if b_j is at the end of a gap of length l ,

0 means there is no similarity up to a_i and b_j .



How do we adapt the algorithm?

Partial Order Alignment

		T	G	T	T	A	C	G	G
		0	0	0	0	0	0	0	0
G		0	0	3	1	0	0	3	3
G		0	0	3	1	0	0	3	6
T		0	3	1	6	4	2	0	4
T		0	3	1	4	9	7	5	3
G		0	1	6	4	7	6	4	8

A	0	0	4	3	5	10	8	6	5
C	0	0	2	1	3	8	13	11	9

C	0	0	2	1	3	8	13	11	9
A	0	0	4	3	5	10	8	6	5

T	0	3	1	5	4	6	11	10	8
A	0	1	0	3	2	7	9	8	7

How do we adapt the algorithm?

Partial Order Alignment

		T	G	T	T	A	C	G	G
		0	0	0	0	0	0	0	0
G		0	0	3	1	0	0	3	3
G		0	0	3	1	0	0	3	6
T		0	3	1	6	4	2	0	4
T		0	3	1	4	9	7	5	3
G		0	1	6	4	7	6	4	8

A	0	0	4	3	5	10	8	6	5
C	0	0	2	1	3	8	13	11	9

C	0	0	2	1	3	8	13	11	9
A	0	0	4	3	5	10	8	6	5

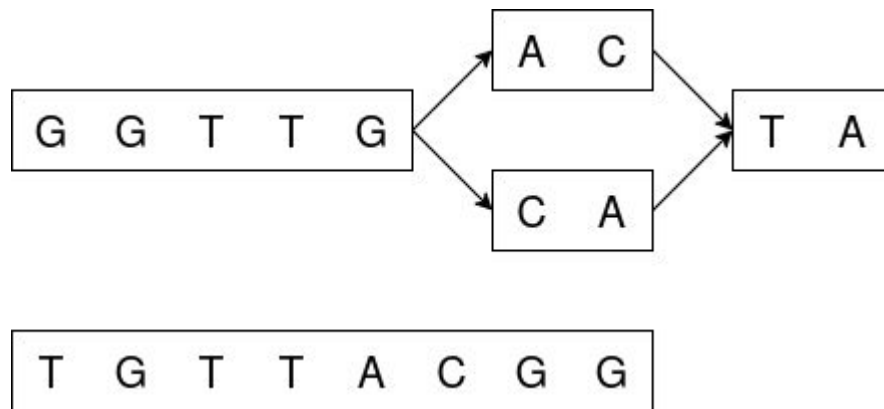
T	0	3	1	5	4	6	11	10	8
A	0	1	0	3	2	7	9	8	7

Diagram illustrating Partial Order Alignment. The top table shows a sequence alignment matrix for sequences GGTGTT and GGTGTT. The bottom left table shows a sequence alignment matrix for sequences A and C. The bottom right table shows a sequence alignment matrix for sequences C and A. Red boxes highlight the value 5 in the bottom left table (row C, column 5) and the value 5 in the bottom right table (row A, column 5). Arrows point from these red boxes to the value 5 in the bottom right table (row T, column 4), indicating a specific alignment path.

Properties of POA

Partial Order Alignment

- The input graph must be a DAG.
- Nodes on the same topological level have no dependencies between them



	T	G	T	T	A	C	G	G
	0	0	0	0	0	0	0	0
G	0							
G	0							
T	0							
T	0							
G	0							
A	0							
C	0							
T	0							
A	0							

	T	G	T	T	A	C	G	G
	0	0	0	0	0	0	0	0
G	0							
G	0							
T	0							
T	0							
G	0							
A	0							
C	0							
T	0							
A	0							

Thread warp A | Thread warp B

Code implementation

CPU Baseline

- 1 thread
- no SIMD
- Tests on Baldo and DGX Spark

Version	Baldo (ms)	DGX Spark (ms)
Baseline	3 903	3 902

CPU Baseline Optimisations

		T	G	T	T	A	C	G	G
	0	0	0	0	0	0	0	0	0
G	0	0	3	1	0	0	0	3	3
G	0	→ 0	→ 3	→ 1	→ 0	→ 0	→ 0	→ 3	→ 6
T	0								
T	0								
G	0								
A	0								
C	0								
T	0								
A	0								

Version	Baldo (ms)	DGX (ms)
Baseline	3 903	3 902

CPU Baseline Optimisations

		T	G	T	T	A	C	G	G
	0	0	0	0	0	0	0	0	0
G	0	0	X						
G	0	X							
T	0								
T	0								
G	0								
A	0								
C	0								
T	0								
A	0								

Version	Baldo (ms)	DGX (ms)
Baseline	3 903	3 902

CPU Baseline Optimisations

		T	G	T	T	A	C	G	G
	0	0	0	0	0	0	0	0	0
G	0	0	X	X					
G	0	X	X						
T	0	X							
T	0								
G	0								
A	0								
C	0								
T	0								
A	0								

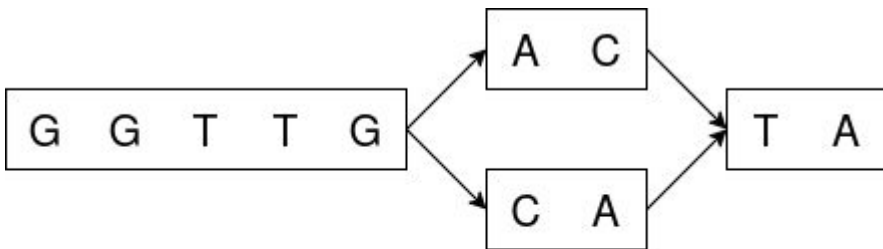
Version	Baldo (ms)	DGX (ms)
Baseline	3 903	3 902

CPU Baseline Optimisations

		T	G	T	T	A	C	G	G
	0	0	0	0	0	0	0	0	0
G	0	0	X	X	X				
G	0	X	X	X					
T	0	X	X						
T	0	X							
G	0								
A	0								
C	0								
T	0								
A	0								

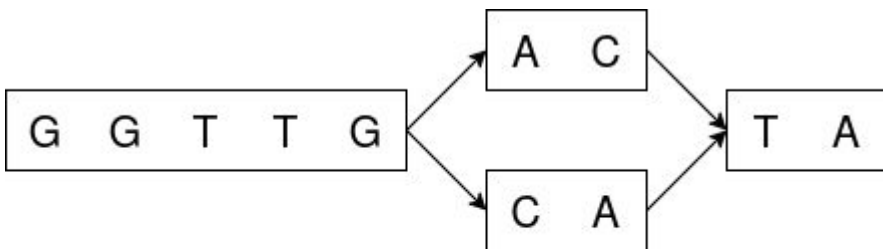
Version	Baldo (ms)	DGX (ms)
Baseline	3 903	3 902
SIMD	974	466

CPU Baseline Optimisations



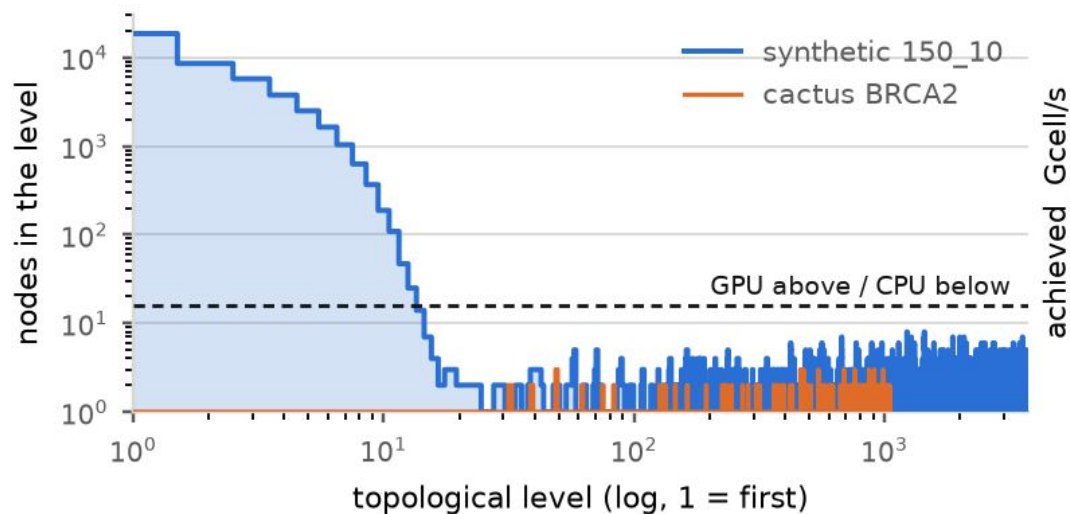
Version	Baldo (ms)	DGX (ms)
Baseline	3 903	3 902
SIMD	974	466

CPU Baseline Optimisations



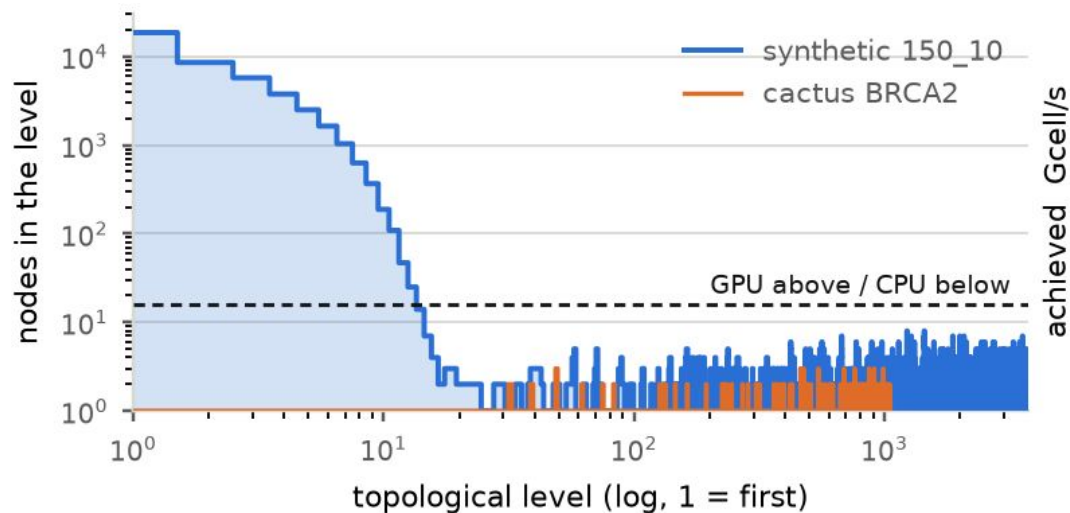
Version	Baldo (ms)	DGX (ms)
Baseline	3 903	3 902
SIMD	974	466
multithread	243	132

CPU Baseline Optimisations



Version	Baldo (ms)	DGX (ms)
Baseline	3 903	3 902
SIMD	974	466
multithread	243	132

CPU Baseline Optimisations



Version	Baldo (ms)	DGX (ms)
Baseline	3 903	3 902
SIMD	974	466
multithread	243	132

Version	Speedup	Speedup
Baseline	1x	1x
SIMD	4x	8,37x
multithread	16x	23x

GPU Baseline

- ❑ Nodes traversed sequentially
- ❑ Uses SIMD approach
- ❑ Tests on Baldo and DGX Spark

Version	Baldo (ms)	DGX Spark (ms)
Baseline CPU	3 903	3 902
Baseline GPU	10 815	8 680

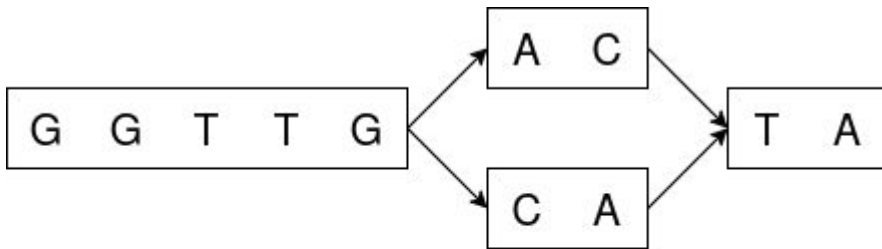
GPU Optimisations

Version	Baldo (ms)	DGX (ms)
Baseline	10 815	8 680

GPU Optimisations

Total	Operation
18,26 GiB	[CUDA memcpy Device-to-Host]
17,70 MiB	[CUDA memcpy Host-to-Device]

Version	Baldo (ms)	DGX (ms)
Baseline	10 815	8 680



GPU Optimisations

Version	Baldo (ms)	DGX (ms)
Baseline	10 815	8 680
Pinned mem & Multiple nodes	1 231	1 071

▼ CUDA HW (0000:01:00.0 - NVIDIA)	Kernel Memory	
▶ 73,7% Kernels		
▶ 26,3% Memory		

GPU Optimisations

Version	Baldo (ms)	DGX (ms)
Baseline	10 815	8 680
Pinned mem & Multiple nodes	1 231	1 071
Async copies	1 019	858

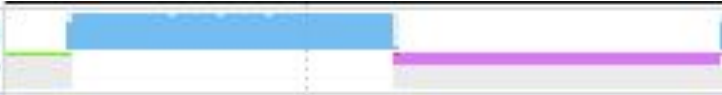
▼ CUDA HW (0000:01:00.0 - NVIDIA)	Kernel Memory	
▶ 73,7% Kernels		
▶ 26,3% Memory		

GPU Optimisations

Version	Baldo (ms)	DGX (ms)
Baseline	10 815	8 680
Pinned mem & Multiple nodes	1 231	1 071
Async copies	1 019	858

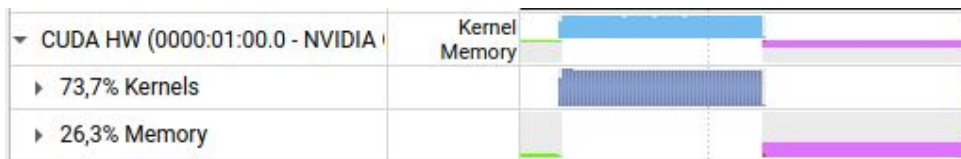
??

??

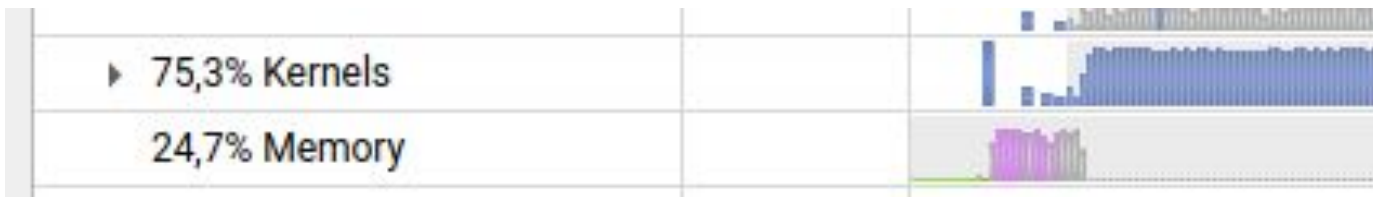
▼ CUDA HW (0000:01:00.0 - NVIDIA)	Kernel Memory	
▶ 73,7% Kernels		
▶ 26,3% Memory		

2x expected?

GPU Optimisations



Too many individual memcopy ops!

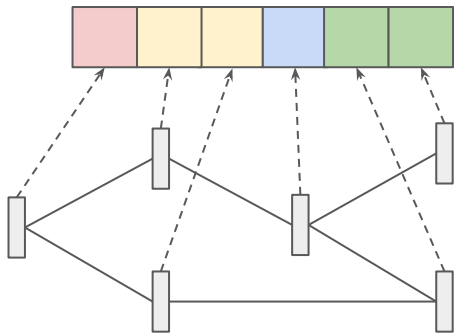
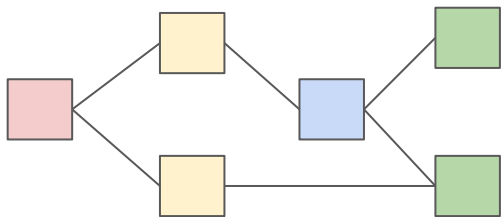


Version	Baldo (ms)	DGX (ms)
Baseline	10 815	8 680
Pinned mem & Multiple nodes	1 231	1 071
Async copies	1 019	858

??

??

GPU Optimisations



Version	Baldo (ms)	DGX (ms)
Baseline	10 815	8 680
Pinned mem & Multiple nodes	1 231	1 071
Async copies	1 026	858
Monolithic	778	724

GPU Optimisations



Now that the trace looks better, only thing left is to improve on the kernel!

Version	Baldo (ms)	DGX (ms)
Baseline	10 815	8 680
Monolithic	778	724

GPU Optimisations



Now that the trace looks better, only thing left is to improve on the kernel!

- Idea: Use shared memory for dp matrix calculation

Version	Baldo (ms)	DGX (ms)
Baseline	10 815	8 680
Batching	778	724

GPU Optimisations

		T	G	T	T	A	C	G	G
	0	0	0	0	0	0	0	0	0
G	0								
G	0								
T	0								
T	0								
G	0								
A	0								
C	0								
T	0								
A	0								

Version	Baldo (ms)	DGX (ms)
Baseline	10 815	8 680
Batching	778	724

□ Problem: Sequences too big

GPU Optimisations

		T	G	T	T	A	C	G	G
	0	0	0	0	0	0	0	0	0
G	0								
G	0								
T	0								
T	0								
G	0								
A	0								
C	0								
T	0								
A	0								

Version	Baldo (ms)	DGX (ms)
Baseline	10 815	8 680
Batching	778	724

□ Solution: Divide and conquer!

GPU Optimisations

		T	G	T	T	A	C	G	G
	0	0	0	0	0	0	0	0	0
G	0								
G	0								
T	0								
T	0								
G	0								
A	0								
C	0								
T	0								
A	0								

Version	Baldo (ms)	DGX (ms)
Baseline	10 815	8 680
Batching	778	724

- Solution: Divide and conquer!
- Problem: DP matrix does not fit in shared

GPU Optimisations

		T	G	T	T	A	C	G	G
	0	0	0	0	0	0	0	0	0
G	0								
G	0								
T	0								
T	0								
G	0								
A	0								
C	0								
T	0								
A	0								

Version	Baldo (ms)	DGX (ms)
Baseline	10 815	8 680
Batching	778	724

- Solution: Divide and conquer!
- Problem: DP matrix does not fit in shared

GPU Optimisations

		T	G	T	T	A	C	G	G
	0	0	0	0	0	0	0	0	0
G	0	0	0	3					
G	0	0	0						
T	0								
T	0								
G	0								
A	0								
C	0								
T	0								
A	0								

Version	Baldo (ms)	DGX (ms)
Baseline	10 815	8 680
Batching	778	724

- Solution: Divide and conquer!
- Problem: DP matrix does not fit in shared

GPU Optimisations

		T	G	T	T	A	C	G	G
G	0	0				0	0	0	0
G	0								
T									
T									
G	0								
A	0								
C	0								
T	0								
A	0								

Version	Baldo (ms)	DGX (ms)
Baseline	10 815	8 680
Batching	778	724

- Solution: Divide and conquer!
- Problem: DP matrix does not fit in shared

GPU Optimisations

		T	G	T	T	A	C	G	G
G	0	0	0	0	0	0	0	0	0
G	0								
T	0								
T	0								
G	0								
A	0								
C	0								
T	0								
A	0								

Version	Baldo (ms)	DGX (ms)
Baseline	10 815	8 680
Batching	778	724

- Solution: Divide and conquer!
- Solution: We only need 3 diagonals!

GPU Optimisations

		T	G	T	T	A	C	G	G
	0	0	0	0	0	0	0	0	0
G	0								
G	0								
T	0								
T	0								
G	0								
A	0								
C	0								
T	0								
A	0								

Version	Baldo (ms)	DGX (ms)
Baseline	10 815	8 680
Batching	778	724
Shared mem	281	336

- Result: shared mem version only has to go to global to load predecessor's score

GPU Optimisations

		T	G	T	T	A	C	G	G
	0	0	0	0	0	0	0	0	0
G	0								
G	0								
T	0								
T	0								
G	0								
A	0								
C	0								
T	0								
A	0								

Version	Baldo (ms)	DGX (ms)
Baseline	10 815	8 680
Batching	778	724
Shared mem	281	336

- Result: shared mem version only has to go to global to load predecessor's score

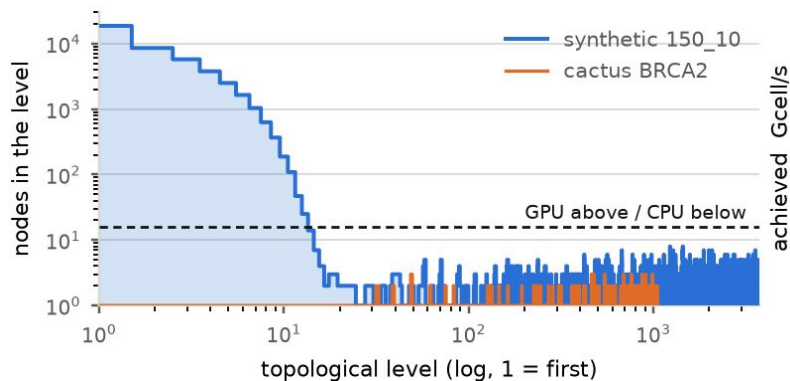
GPU Optimisations

		T	G	T	T	A	C	G	G
	0	0	0	0	0	0	0	0	0
G	0								
G	0								
T	0								
T	0								
G	0								
A	0								
C	0								
T	0								
A	0								

Version	Baldo (ms)	DGX (ms)
Baseline	10 815	8 680
Batching	778	724
Shared mem	281	336

- Result: shared mem version only has to go to global to load predecessor's score

GPU Optimisations



Version	Baldo (ms)	DGX (ms)
Baseline	10 815	8 680
Batching	778	726
Shared mem	395	336
CPU best	243	132



- We could optimise more the kernel, but exec time is already on nodes where the GPU is worse than CPU

Hybrid versions

- Assign narrow levels to CPU, and wider levels to GPU
- Copies data from CPU-GPU asynchronously

Version	Baldo (ms)	DGX Spark (ms)
GPU best	395	336
CPU best	243	132
Hybrid base	255	150

Hybrid Optimisations

- DGX Spark has physically shared memory, use managed memory to avoid copies altogether

Version	Baldo (ms)	DGX (ms)
GPU best	395	336
CPU best	243	132
Hybrid base	275	178

Hybrid Optimisations

- Managed mem is avoiding copies, but pinned is better, no?

Version	Baldo (ms)	DGX (ms)
GPU best	395	336
CPU best	243	132
Hybrid base	275	178
Managed mem	544	105

Hybrid Optimisations

- Pinned memory is bad?

Version	Baldo (ms)	DGX (ms)
GPU best	395	336
CPU best	243	132
Hybrid base	275	178
Managed mem	544	105
Pinned mem	346	120 ??

Hybrid Optimisations

- Pinned memory is bad?
- Yes, for some reason it's 14% worse, although I couldn't find out why

Version	Baldo (ms)	DGX (ms)
GPU best	395	336
CPU best	243	132
Hybrid base	275	178
Managed mem	544	105
Pinned mem	346	120 ??

Hybrid Optimisations

- ☐ Pinned memory is bad?
- ☐ Yes, for some reason it's 14% worse, although I couldn't find out why
- ☐ On baldo copies take so long the GPU->CPU switch is not worth it.

Version	Baldo (ms)	DGX (ms)
GPU best	395	336
CPU best	243	132
Hybrid base	275	178
Managed mem	544	105
Pinned mem	346	120 ??

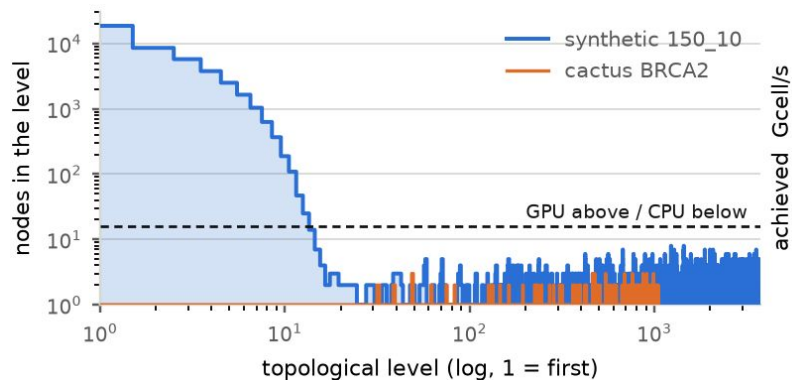


Last column optimisation

- Only keep the last column, recalculate DP matrix during traceback

Version	Baldo (ms)	DGX (ms)
Baseline CPU	3 903	3 902
Multithread + SIMD	243	132
Last col CPU	231	91
Last col GPU	400	237

Last column GPU kernels



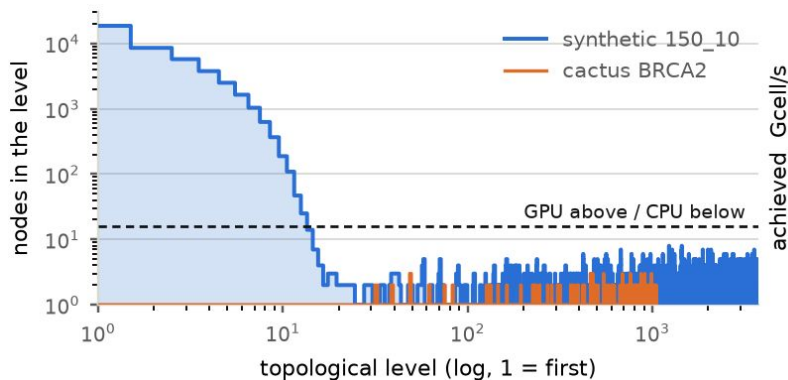
Version	Baldo (ms)	DGX (ms)
Last col GPU	400	237
Last col CPU	231	90
Last col hybrid	141	64

Total	Operation
18,26 GiB	[CUDA memcpy Device-to-Host]
17,70 MiB	[CUDA memcpy Host-to-Device]

Total	Operation
117,68 MiB	[CUDA memcpy Device-to-Host]
17,70 MiB	[CUDA memcpy Host-to-Device]

Last column GPU kernels

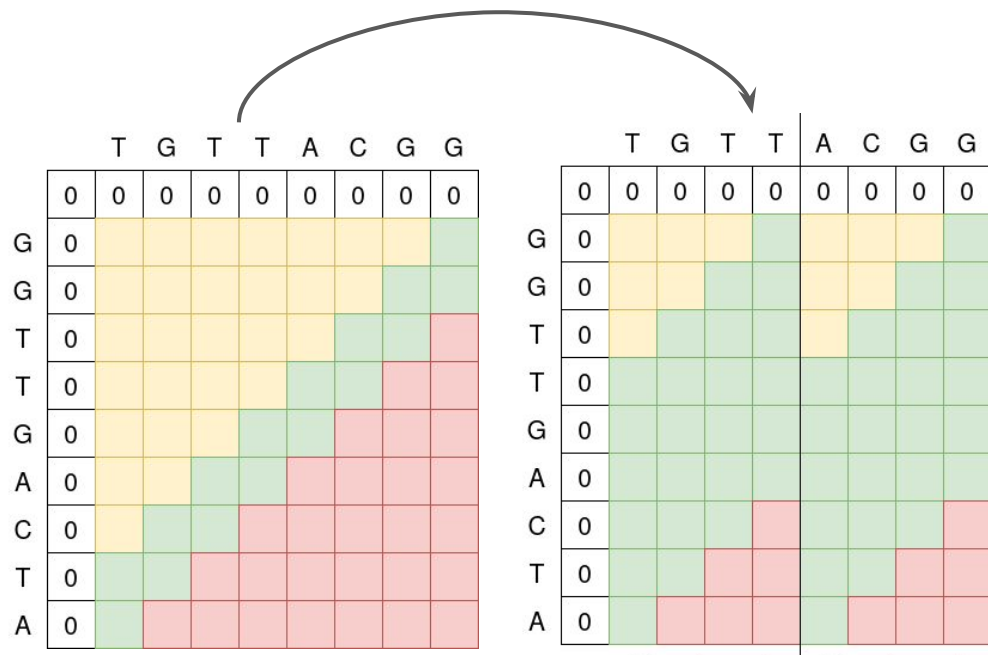
- Measure the performance of GPU kernels inside dense region, levels with ≥ 16 nodes, and top 3 dense nodes for best-case



Version (≥ 16)	Baldo (Gcells/s)	DGX (Gcells/s)
Last col GPU	75.28	124.36

Version (top 3)	Baldo (Gcells/s)	DGX (Gcells/s)
Last col GPU	79	131

Last column GPU kernels



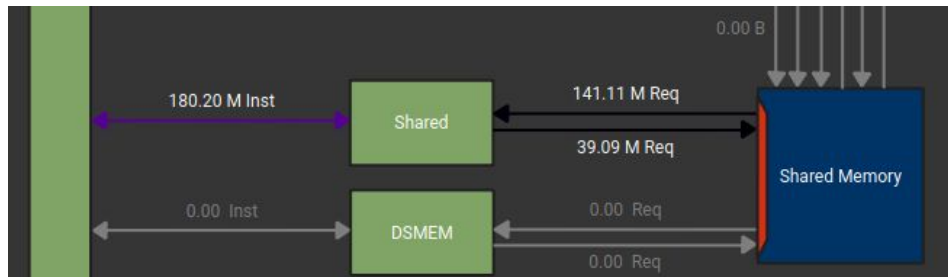
Version (>= 16)	Baldo (Gcells/s)	DGX (Gcells/s)
Last col GPU	75	124
Warps	109	161

Version (top 3)	Baldo (Gcells/s)	DGX (Gcells/s)
Last col GPU	79	131
Warps	130	184

Last column GPU kernels

- We are maxing out shared memory bandwidth, at 96% throughput

ID	Function Name	Compute Throughput	Memory Throughput [
0	compute_dp_gpu_...	96,38	96,38
1	compute_dp_gpu_...	92,87	92,87

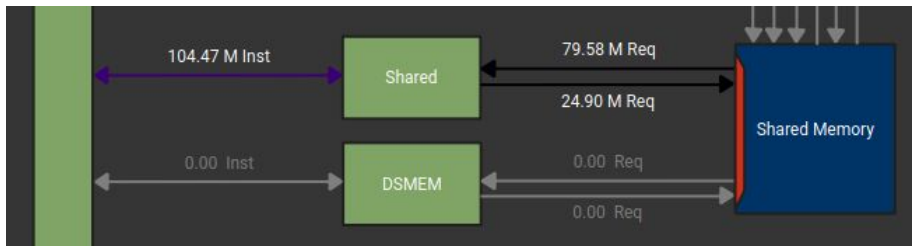
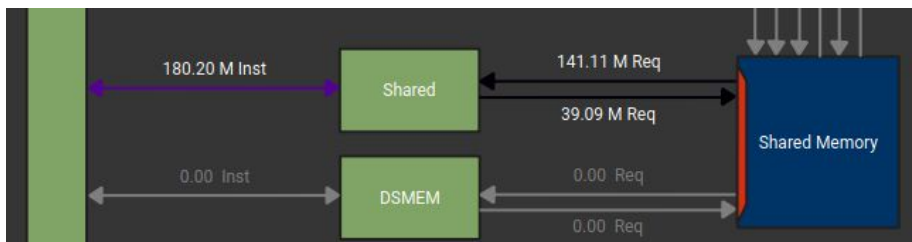


Version (>= 16)	Baldo (Gcells/s)	DGX (Gcells/s)
Last col GPU	75	124
Warps	109	161
Registers	128	150

Version (top 3)	Baldo (Gcells/s)	DGX (Gcells/s)
Last col GPU	79	131
Warps	130	184
Registers	162	178

Last column GPU kernels

- Use registers! 180M -> 104M req

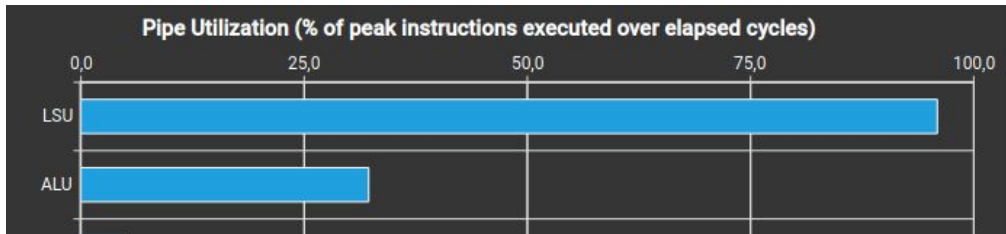


Version (>= 16)	Baldo (Gcells/s)	DGX (Gcells/s)
Last col GPU	75	124
Warps	109	161
Registers	128	150

Version (top 3)	Baldo (Gcells/s)	DGX (Gcells/s)
Last col GPU	79	131
Warps	130	184
Registers	162	178

Last column GPU kernels

- Use registers! 180M -> 104M req
- Registers introduces many new __shfl primitives to move data between threads, which now saturates LSU units



Version (>= 16)	Baldo (Gcells/s)	DGX (Gcells/s)
Last col GPU	75	124
Warps	109	161
Registers	128	150

Version (top 3)	Baldo (Gcells/s)	DGX (Gcells/s)
Last col GPU	79	131
Warps	130	184
Registers	162	178

Last column GPU kernels

- Reduce LSU workload by batching memory requests, however, this uses more registers
- Occupancy is now the limiting factor, due to high register usage (56 -> 72 registers per thread)

Version (>= 16)	Baldo (Gcells/s)	DGX (Gcells/s)
Last col GPU	75	124
Warps	109	161
Registers	128	150
Merged_req	115	162

Version (top 3)	Baldo (Gcells/s)	DGX (Gcells/s)
Last col GPU	79	131
Warps	130	184
Registers	162	178
Merged_req	143	195

Last column GPU kernels

- Reduce LSU workload by batching memory requests, however, this uses more registers
- Occupancy is now the limiting factor, due to high register usage (56 -> 72 registers per thread)

Version (>= 16)	Baldo (Gcells/s)	DGX (Gcells/s)
Last col GPU	1x	1x
Warps	1,45x	1,29x
Registers	1,7x	1,21x
Merged_req	1,53x	1,31x

Version (top 3)	Baldo (Gcells/s)	DGX (Gcells/s)
Last col GPU	1x	1x
Warps	1,64x	1,4x
Registers	2,05x	1,36x
Merged_req	1,81x	1,49x

New best hybrid versions

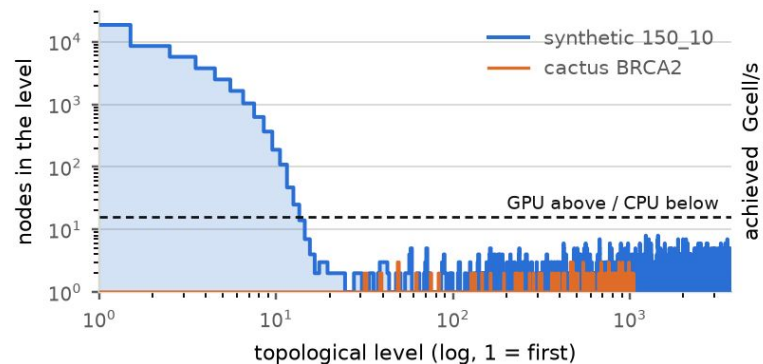
Version	Baldo (ms)	DGX (ms)
Baseline CPU	3 903	3 902
Last col CPU	231	91
Hybrid best	139	64

Final speedups

Version	Baldo (ms)	DGX (ms)
Baseline CPU	3 903	3 902
Last col CPU	16,9x	42,9x
Hybrid best	28x (1,65x)	61x (1,42x)

SOTA Comparison (HGA)

- Using a real dataset, for BRCA2 pangenome graph
- Usin best GPU kernel for each, and extracting parallelism from multiple reads instead of nodes in the same topological level



Version	Baldo (GCUPS)	DGX (GCUPS)
HGA (1 read)	0.01	0.01
HGA (1024 reads)	5.3	7.6
HGA (8704 reads)	43.7	32.9
GPU Best (1 read)	0.16	0.27
GPU Best (64 reads)	9.5	15
GPU Best (1024 reads)	55.7	49.5

Any questions?

CPU Baseline Optimisations

		T	G	T	T	A	C	G	G
	0	0	0	0	0	0	0	0	0
G	0								
G	0								
T	0								
T	0								
G	0								
A	0								
C	0								
T	0								
A	0								

Version	Baldo (ms)	DGX (ms)
Baseline	3 903	3 902
SIMD	974	466

GPU Optimisations

Version	Baldo (ms)	DGX (ms)
Baseline	10 815	8 680
Pinned mem & Multiple nodes	1 231	1 071
Async copies	1 019	858
Monolithic	788	726



Looks better, but...
Command buffer is full,
compute has to wait

GPU Optimisations



- Thanks to monolithic, accumulate many smaller level's memcpy until mem requests are big enough.

Version	Baldo (ms)	DGX (ms)
Baseline	10 815	8 680
Pinned mem & Multiple nodes	1 231	1 071
Async copies	1 019	858
Monolithic	778	724

GPU Optimisations



- Thanks to monolithic, accumulate many smaller level's memcpy until mem requests are big enough.

Version	Baldo (ms)	DGX (ms)
Baseline	10 815	8 680
Pinned mem & Multiple nodes	1 231	1 071
Async copies	1 019	858
Monolithic	788	726
Batching	778	724

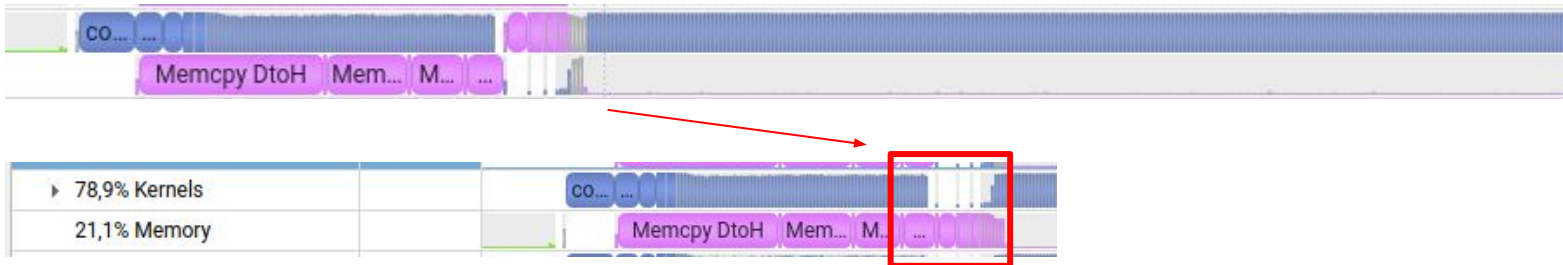
??

??

GPU Optimisations

This only happened on my own system, with a modest GPU (5060Ti), batching solved it, with a 1,15x speedup

Version	Baldo (ms)	DGX (ms)
Baseline	10 815	8 680
Pinned mem & Multiple nodes	1 231	1 071
Async copies	1 026	858
Monolithic	788	726
Batching	778	724



What about global memory writes?

Use more buffers! Until the latency is hidden behind the throughput

		T	G	T	T	A	C	G	G
	0	0	0	0	0	0	0	0	0
G	0								
G	0								
T	0								
T	0								
G	0								
A	0								
C	0								
T	0								
A	0								

Last column GPU kernels

- Time is not useful measurement anymore, because of the long tail, and now hybrid versions work well even on discrete systems, due to the reduction in data movement required

Dense levels,
containing 80% of
cell updates

Thin levels, containing 14% of total work

Version	Baldo (ms)	DGX (ms)
Last col GPU	400	237
Last col CPU	231	90
Last col hybrid	141	64



Total	Operation
18,26 GiB	[CUDA memcpy Device-to-Host]
17,70 MiB	[CUDA memcpy Host-to-Device]

Total	Operation
117,68 MiB	[CUDA memcpy Device-to-Host]
17,70 MiB	[CUDA memcpy Host-to-Device]